

5. Exercise: Benchmarking and Production Ready System

name(s): _____ points: _____ (max. 24)

In this exercise, you will use your optimized gesture-recognition model from Exercise 4 on the Raspberry Pi. You will deploy an application with two modes. Mode one continually classifies what gestures are being drawn by the user. Mode two is a benchmarking mode that you will use to determine the performance of your finally optimized model.

- The code must be **your own work**.
- You may base your code on the code from the in class lessons.
- You may use online resources to understand TensorFlow, TFLite and optimization techniques.
- You must not use generative AI tools to write the code for you.
- You must be able to explain your code and design decisions in the lesson.
- **Failure to understand / explain** your own code and design decisions can lead to **negative grading**.

1. Part *Create a fully optimized model.* 3 points

- Use the optimization code from *Exercise 4* to create a model that combines neuron pruning, channel pruning and int8 quantization (only use whatever pruning methods can be applied to your model, e.g. no channel pruning necessary if you don't use a CNN architecture).
- Tune the parameters when pruning to achieve acceptable results in terms of accuracy.
- Export the unoptimized baseline model to a TFLite file.

2. Part *Adapt the C++ code structure.* 15 points

- Orient yourself on the *TfliteDigitClassifier*, to expand your *TfliteGestureClassifier*.
- The *TfliteGestureClassifier* should support a benchmarking mode to benchmark the models.
- The *TfliteGestureClassifier* should also support a production ready operation mode that classifies gestures on a continuous data stream from the sensor.
- If the garbage class is detected, the LED matrix should show no output. If your model recognizes a gesture, a corresponding symbol shall be shown on the matrix e.g. a, b, 1, 2, arrows, etc. To show the digits you can use

and modify the class *SenseHatDisplay*.

- (e) To correctly classify gestures on a data stream, we suggest using a sliding window over the data where a gesture is being recognized if a number of overlapping sliding windows have been recognized as the same gesture.

3. Part *Test your model on continuous data.*

3 points

- (a) Validate that your application can classify gestures from different people when drawing with it as intended.
- (b) Note down if you find any issues in the operation of the device (e.g. recognizes gestures where there are none, misclassifies during a gesture, and gets gestures mixed up).
- (c) Your model does not have to work perfectly. If your model has significant issues, be prepared to discuss them during the lecture. What do you think causes these problems?

4. Part *Benchmark your model.*

3 points

- (a) Run your application in benchmark mode to determine performance improvement when using the optimized model compared to the original.
- (b) For ease of use, we strongly suggest using the scripting from the in class lessons for benchmarking (*perf* target).
- (c) Note down the size of the baseline TFLite file vs the optimized TFLite file.
- (d) Note down the ms per inference for both the baseline and optimized models.
- (e) Additionally, you can include the other models you created in *Exercise 4* in your benchmark. This is fully optional and not required.